

Sieci komputerowe- Projekt

Rok akademicki	Semestr	Kierunek	Sekcja	Data
2025/2026	4	IPpp	3	18.09.2026

Skład sekcji:

Natalia Kuśka,

Daniel Byra,

Kacper Bieniek,

Podział obowiązków

Daniel Byra:

- Weryfikacja oraz walidacja aplikacji zgodnie z wymaganiami PK oraz SK.
- Zaprojektowanie protokołu sieciowego.

Kacper Bieniek:

- Implementacja protokołu sieciowego.
- Przygotowanie testów integracyjnych.

Natalia Kuśka:

- Implementacja komunikacji dla trybu jedno oraz obustronnego.
- Dostosowanie elementów GUI pod nowe funkcjonalności związane z komunikacją sieciową zgodnie z wytycznymi projektowymi.

Działanie protokołu komunikacyjnego

Komunikacja wykorzystuje protokół TCP (QTcpSocket, QTcpServer). Aplikacja posiada warstwę protokołu (communication_protocol) oraz węzeł sieciowy (network_hub).

Struktura ramki

Nagłówek 12 bajtów:

- typ wiadomości 4 bajty,
- rozmiar danych 4 bajty,
- numer próbki 4 bajty,

Dane (symulacja / sterownie / konfiguracja):

- symulacja (sygnał_sterujący;wartość_regulowana;wartość_zadana;czas).

Przykład: 1.2;2.5;.3.1;10

- sterowanie (polecenie;interwał;okno).

Przykład: START;50;100

- konfiguracja (ustawienia_pid|ustawienia_arx|ustawienia_generatora).

Przykład: Kp=1.1,Ti=10,Td=0.1|k=1,dA=1.1,dB=0.4|typ=skok,amp=2.0

Każda wiadomość zawiera 12-bajtowy nagłówek binarny oraz payload czyli sformatowany jako ciąg znaków string.

Parser strumienia przetworz_strumien(QByteArray &surowe_dane):

1. Sprawdza, czy bufor zawiera co najmniej 12 bajtów nagłówka. Jeśli nie, wstrzymuje odczyt i czeka na kolejne dane z TCP.
2. Odczytuje pola typ, rozmiar i numer_probki za pomocą QDataStream.
3. Sprawdza obecność całego pakietu (12 + rozmiar) w buforze. Jeśli dane są niekompletne, przerywa działanie bez usuwania bajtów.
4. Po skompletowaniu wycina ładunek i usuwa przetworzone bajty z bufora wejściowego.
5. Emituje sygnał Qt ze zdeserializowaną strukturą danych.

Tryby pracy sieciowej:

1. Tryb Jednostronny:

- Taktowanie wyznacza wyłącznie zegar regulatora (MainWindow::zegar).
- W każdym takcie regulatora oblicza u , wysyła ramkę do obiektu i odkłada snapshot {czas, W} do kolejki FIFO m_sentFramesQueue.
- obiekt po odebraniu pakietu wykonuje krok symulacji ARX i natychmiast odsyła y .
- regulator pobiera skojarzony czas oraz wartość zadaną z kolejki FIFO i rysuje punkt na wykresie.

2. Tryb Obustronny:

- Obie instancje posiadają niezależne taktowanie i buforowanie.

- Regulator zapisuje swój stan w `m_masterBuffer` pod numerem pakietu.
- Obiekt reaguje od razu na dane z sieci (opóźnienie poniżej 2 ms).
- W tle działa timer zapasowy obiektu.
- Przy opóźnieniach obiekt pobiera dane z kolejki `m_slaveBuffer`. Gdy bufor jest pusty, przechodzi w tryb awaryjny – utrzymuje ostatnią wartość która została przekazana przez protokół.

Rozwój aplikacji

Etap 0: Walidacja oraz weryfikacja kodu bazowego:

- Zidentyfikowano oraz wyeliminowano błędy pozostawione w projekcie przez poprzednią sekcję.

Etap I: Architektura protokołu i warstwa TCP:

- Zaprojektowano protokół komunikacyjny w klasie `communication_protocol`, definiując 12-bajtowy nagłówek binarny oraz identyfikatory typów wiadomości.
- Oparto warstwę transportową na komponentach `QTcpSocket` i `QTcpServer` w ramach modułu `network_hub`.
- Zaimplementowano parser strumieniowy `przetworz_strumien`, eliminujący fragmentację oraz zjawisko sklejania pakietów TCP.

Etap II: Wdrożenie trybu jednostronnego:

- Utworzono klasę dla trybu Jednostronnego zarządzając transmisją asynchroniczną.
- Przyjęto architekturę: węzeł regulatora determinuje taktowanie zegara, natomiast obiekt odpowiada na napływające ramki danych.
- Wprowadzono licznik brakujących odpowiedzi (`m_missedPacketsCount`) w celu detekcji ubytków pakietów.

Etap III: Dostosowanie interfejsu graficznego (GUI) pod funkcjonalności sieciowe:

- Dodano zestaw ikon sygnalizacyjnych stan połączenia oraz ikony zmieniające tryb pracy (sieciowy, stacjonarny) aplikacji, wybór serializacji (binarna, tekstowa), obiektu lub regulatora oraz trybu taktowania. Wprowadzono również informacje związane z nawiązaniem połączenia oraz kontrolą pakietów.

Etap IV: Wdrożenie trybu obustronnego:

- Utworzono klasę dla trybu Obustronnego do obsługi symulacji rozproszonej.
- Zaimplementowano niezależne taktowanie: rozdzielono zegary na timer regulatora oraz niezależny timer po stronie węzła obiektu.
- Wprowadzono buforowanie stanów przy użyciu struktur `m_masterBuffer` oraz `m_slaveBuffer`.

Etap V: Eliminacja wyścigów czasowych i kompensacja RTT:

- Zastąpiono pojedyncze zmienne rejestrujące buforem implementującym zasadę FIFO, uzyskując precyzyjne dopasowanie czasowe próbek.

Etap VI: Odporność na rekonfigurację i przełączanie trybów:

- Uporządkowano sekwencję niszczenia timerów oraz zamykania gniazd TCP przy dynamicznej zmianie trybu pracy.
- Dodano przeliczanie próbek oraz czasu, które sprawiają, że po przełączeniu trybów nie dochodzi do cofania się wykresów oraz nie ma skoków różnych wartości.

Etap VII: Poprawa działania symulacji:

- Wdrożono mechanizm bezuderzeniowego przełączania w metodzie `dostosujCalkeDoU`, niwelujący skoki sygnału sterującego po ponownym nawiązaniu połączenia.
- Zsynchronizowano zakres osi czasu na wykresach oraz przyciski START, STOP i RESET.
- Zastąpiono blokujące okna dialogowe asynchronicznym systemem powiadomień, gwarantując ciągłość pracy pętli zdarzeń aplikacji.

Napotkane trudności

- Sklejanie i fragmentacja TCP
 - **Problem:** Znalezienie “złotego środka” pomiędzy wielkością wysydanego pakietu, a jego prędkością.
 - **Rozwiązanie:** Ułożenie odpowiedniej struktury ramki pakietu.

- Wyścigi czasowe
 - **Problem:** Wartość zadana wyprzedzała wyjście o 1-2 kroki na wykresie. Odpowiedź z sieci wracała z opóźnieniem.
 - **Rozwiązanie:** Dodanie kolejki FIFO, zostaje tam zapisany czas oraz wartość zadana. Po dotarciu odpowiedzi z opóźnieniem program pobierał dane dla starszych wartości, a nie dla najnowszej.
- Stany nieustalone przy zmianie interwału
 - **Problem:** Podczas zmiany interwału (np. z 200ms na 100ms) dochodziło do nagłego wyzerowania części całkującej, która po czasie dochodziła do zadanej wartości.
 - **Rozwiązanie:** Zostało wprowadzone odpowiednie równanie, które wylicza całkę, powoduje to brak skoków.
- Blokada ramek po zmianie interwału
 - **Problem:** Po zmianie interwału dochodziło do odrzucania pakietów przez początkowe sekundy. Aplikacja odrzucała pakiety, które przyszły za późno.
 - **Rozwiązanie:** Zamiast ustawienia stałej liczby próbek, limit zostaje wyliczany tak, aby odpowiadał co najmniej 2 sekundom bufora sieciowego.
- Blokowanie komunikacji przez okna modalne
 - **Problem:** Okienka służące do informowania użytkownika o nawiązaniu połączenia lub rozłączeniu ingerowały w wątek, co powodowało, że dane nie dochodziły i symulacja miała braki.
 - **Rozwiązanie:** Wdrożono komunikaty działające asynchronicznie.

Podsumowanie czego nauczył się każdy członek sekcji

Kacper Bieniek:

- Poznanie stopnia zaawansowania oraz możliwej różnorodności implementacji protokołu sieciowego w zależności od potrzeb oraz konieczność manipulowania balansem pomiędzy szybkością protokołu a jego pojemnością.
- Zapoznanie się z różnymi sposobami zaznajamiania się z dokumentacją w tym przypadku framework'a Qt w segmencie sieciowym np. stack overflow , modele AI, doc.qt.io

Daniel Byra:

- Umiejętność projektowania autorskiego protokołu sieciowego za pomocą dostępnych rozwiązań udostępnionych przez środowisko QT
- Umiejętność pisania własnych scenariuszy testowych w celu weryfikacji poprawności działania aplikacji na linii użytkowej oraz sieciowej

Natalia Kuśka:

- Zrozumienie różnic oraz działania trybów jedno- oraz obustronnego, późniejsza ich implementacja w kodzie
- Łączenie elementów GUI z funkcjonalnościami sieciowymi
- Zapoznanie się z działaniem modułu sieciowego dla Qt, takich jak np. utworzenie połączenia pomiędzy instancjami, sposobu przesyłania danych